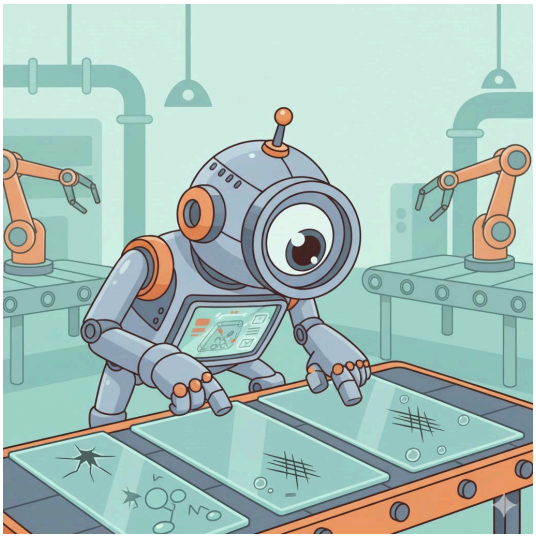
ZURICH UNIVERSITY OF APPLIED SCIENCES
DEPARTEMENT LIFE SCIENCES AND FACILITY MANAGEMENT

Glass Defect Detection: Evaluating Object Detection Models



Projectwork 2

by
Florian Vollmer

Bachelors 2023
Submission date: 2025-12-11
Study Direction: Applied Digital Life Science

Supervisors:
Dr. Stefan Glüge
ZHAW Life Sciences und Facility Management, Wädenswil

Imprint

Recommended Citation:

Florian Vollmer (2025). *Glass Defect Detection: Evaluating Object Detection Models*

.

Keywords: Deep Learning, Object Detectors, Evaluation Metrics, Digital Lab and Production, Factory Production, Image Processing

Abstract

English Version

This project addressed the challenge of automated glass defect detection in the pharmaceutical sector, where the integrity of packaging vessels play an important role. The study quantitatively compared two different object detection architectures: the efficient YOLO Nano (CNN-based Single-Shot Detector) and the more resource-intensive RF-DETR Nano (Transformer-based), utilizing two discrete datasets representing simple and complex, real-world conditions. The results revealed a significant architectural difference: The YOLO model demonstrated overwhelming training efficiency (up to 70% faster than RF-DETR) and achieved superior Precision (0.869 vs. 0.776 on Dataset A), indicating a lower rate of costly false positives. However, the critical finding was the failure to meet safety standards: The best achieved recall was only 0.799 on the complex dataset. This unacceptable rate of false negatives (missed defects) precludes both models from safe industrial deployment. The study concludes that, while the YOLO Nano architecture is the most efficient choice among the tested models, the primary bottleneck is not architectural speed, but the lack of model robustness required to achieve the near-perfect recall necessary to guarantee product safety.

German Version

Dieses Projekt befasste sich mit der Herausforderung der automatisierten Erkennung von Defekten auf Glasoberflächen im Pharmasektor, wo die Unversehrtheit von Verpackungsbehältern eine wichtige Rolle spielt. Die Studie verglich quantitativ zwei verschiedene Architekturen zur Objekterkennung: das effiziente YOLO Nano (CNN-basierter Single-Shot-Detektor) und das ressourcenintensivere RF-DETR Nano (Transformer-basiert), wobei zwei unterschiedliche Datensätze verwendet wurden, die einfache und komplexe reale Bedingungen repräsentieren. Die Ergebnisse zeigten einen signifikanten Unterschied hinsichtlich der Architektur: Das YOLO-Modell zeigte eine überwältigende Trainingseffizienz (bis zu 70 % schneller als das RF-DETR) und erzielte eine überlegene Präzision (0,869 gegenüber 0,776 im Datensatz A), was auf eine geringere Rate kostspieliger Fehlalarme hindeutet. Die entscheidende Erkenntnis war jedoch die Nichteinhaltung der Sicherheitsstandards: Die beste erzielte Recall-Rate betrug nur 0,799 im komplexen Datensatz. Diese inakzeptable Rate an False Negatives (übersehene Fehler) schließt beide Modelle für einen sicheren industriellen Einsatz aus. Die Studie kommt zu dem Schluss, dass die YOLO Nano-Architektur zwar die effizienteste Wahl unter den getesteten Modellen ist, der Hauptengpass jedoch nicht die Geschwindigkeit der Architektur ist, sondern die mangelnde Robustheit des Modells, die erforderlich ist, um die nahezu perfekte Recall-Rate zu erreichen, die zur Gewährleistung der Produktsicherheit notwendig ist.

Table of Contents

List of Abbreviations 4
1. Introduction 5
   1.1. Context and Motivation 5
   1.2. Overall Concept 5
   1.3. Research Questions 6
   1.4. State of the Art 6
2. Methodology 10
   2.1. Datasets and Assumptions 10
   2.2. Model Selection 13
   2.3. Model Mechanics 14
   2.4. Model Training 15
   2.5. Model Testing 15
   2.6. Evaluation Metrics 16
3. Results 18
   3.1. Model Results of Training 18
   3.2. Model Results and Statistical Evaluation of Testing 24
   3.3. Sample Images of Testing 28
4. Discussion 31
   4.1. Interpretation of Results 31
   4.2. Assumptions & Research Questions 34
5. Conclusion 35
   5.1. Summary of Key Findings 35
   5.2. Limitations and Challenges 36
   5.3. Future Research Directions 37
6. Declaration on AI Assistance 38
7. References 39
8. Lists 40
   List of Figures 40
   List of Tables 40
9. Attachments 42
   9.1. Raw Data 42

List of Abbreviations

- **AI**: Aritficial Intelligence
- **AP**: Average Precision
- **CNN**: Convolutional Neural Network
- **COCO**: Common Objects in Context
- **mAP**: Mean Average Precision
- **RF-DETR**: Roboflow Detection Transformer
- **YOLO**: You Only Look Once

1. Introduction

1.1. Context and Motivation

Within the industrial sector, quality control represents a critical success factor. Specifically in pharmaceuticals and medical technology, the integrity of the primary packaging material (typically glass containers) is crucial for product safety.

The primary material used for medical and chemical glass containers generally consists of borosilicate glass. However, this material can exhibit or contain production-related defects, including particulate inclusions, air bubbles, or flexible fragments known as Lamellae [1].

The relevance of these quality deficiencies is highlighted by the consequences demonstrated by [1]:

"In recent years, more than 20 product recalls have been associated with glass issues, and over 100 million units of drugs packaged in vials or syringes have been withdrawn from the market."

This underscores the urgent necessity of reliable detection methods for such flaws.

The rapid advancement of Artificial Intelligence (AI) and fundamental mechanisms like deep learning allows for the application of object detectors for the automated recognition of the aforementioned defects. The attractiveness of these AI-based image recognition methods is continuously increasing [2]. However, as these procedures still represent a nascent field of research, comprehensive validation studies on their suitability and the achievability of the necessary performance in industrial use are lacking.

A central challenge in training robust models is the scarcity of extensive datasets [2]. Specifically, the naturally limited availability of data from defective products, caused by the significant imbalance in favor of intact units, complicates the creation of an adequate training dataset [2].

This work serves as an exploratory contribution to the evaluation of the current feasibility of AI-supported defect detection. The objective is the comparison of available models on the market in order to provide a well-founded statement on the realisability of precise defect determination.

1.2. Overall Concept

The overall objective of this work was to conduct a quantitative performance comparison of two different object detection models on two different datasets. The experimental procedure was based on the following fundamental steps:

1. **Model selection and preparation:** Two specific, pre-trained foundation models from the class of object detectors were selected and initialized for evaluation.

2. **Data management and fine-tuning:** To test the models' generalizability, they were each finetuned to the specific classification task (defect detection) using two discrete datasets (dataset A and dataset B). Cross Validation was applied to ensure statistical robustness.

3. **Performance evaluation:** The performance of the four trained model-dataset combinations was then evaluated on a separate, previously unseen test dataset. The performance was determined based on established quantitative metrics for object detection.

4. **Comparative analysis:** Finally, the generated results were visualized and subjected to a detailed comparative analysis. The aim was to identify the robustness, strengths, and weaknesses of the model architectures examined in relation to glass defect detection.

1.3. Research Questions

Based on the challenges mentioned in **Context and Motivation**, this study tries to answer following research questions:

- How does the performance differ between the two investigated object detection models (Model X vs. Model Y), measured by Mean Average Precision (mAP), in detecting defects in the different datasets?
- Which performance benchmarks (e.g., minimum precision and recall values) must be achieved to deem the resulting reliability of the models as sufficient for deployment in real-world quality control scenarios?

1.4. State of the Art

The quote from [3] explains quite well what Object detection is: *"Object detection is a critical capability of computer vision that identifies and locates objects within an image or video. Unlike image classification, object detection not only classifies the objects in an image, but also identifies their location within the image by drawing a bounding box around each object."*

Given the lack of specific literature concerning the performance of contemporary object detectors on glass vial defect inspection, this study must rely on fundamental model performance benchmarks established by previous research. These benchmarks are typically derived from models trained on broad, general-purpose datasets.

The following analysis focuses on the existing comparative statistics for the chosen architectures: YOLO and RF-DETR.

Model	Size	# Params.	GFLOPS	Latency (ms)	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
Real-Time Object Detection w/ NMS										
YOLOv8 (Jocher et al., 2023)	N	3.2M	8.7	2.1	35.2	49.2	38.3	15.8	38.8	51.3
YOLOv11 (Jocher et al., 2024)	N	2.6M	6.5	2.2	37.1	51.6	40.4	17.3	40.7	55.6
YOLOv8 (Jocher et al., 2023)	S	11.2M	28.6	2.9	42.4	57.6	46.0	22.2	47.1	59.6
YOLOv11 (Jocher et al., 2024)	S	9.4M	21.5	3.2	44.1	59.3	47.9	26.1	48.5	62.6
YOLOv8 (Jocher et al., 2023)	M	25.9M	78.9	5.4	47.3	62.5	51.5	27.5	52.9	65.1
YOLOv11 (Jocher et al., 2024)	M	20.1M	68.0	5.1	48.3	63.6	52.5	29.1	53.8	66.3
Open-Vocabulary Object Detection (Fully-Supervised Fine-Tuning)										
GroundingDINO (Liu et al., 2023)	T	173.0M	1008.3	427.6*	58.2	-	-	-	-	-
End-to-End Real-Time Object Detection										
LW-DETR (Chen et al., 2024a)	T	12.1M	21.4	1.9	42.9	60.7	45.9	22.7	47.3	60.0
D-FINE (Peng et al., 2024)	N	3.8M	7.3	2.1	42.7	60.2	45.4	22.9	46.6	62.1
RF-DETR (Ours)	N	30.5M	31.9	2.3	48.0	67.0	51.4	25.2	53.5	70.0
LW-DETR (Chen et al., 2024a)	S	14.6M	31.8	2.6	48.0	66.8	51.6	26.7	52.5	65.6
D-FINE (Peng et al., 2024)	S	10.2M	25.2	3.5	50.6	67.6	55.0	32.6	54.6	66.6
RF-DETR (Ours)	S	32.1M	59.8	3.5	52.9	71.9	57.0	32.0	58.3	73.0
RT-DETR (Zhao et al., 2024)	R18	36.0M	100.0	4.4	49.0	66.6	53.3	32.8	52.1	65.0
LW-DETR (Chen et al., 2024a)	M	28.2M	83.9	4.4	52.6	72.0	56.6	32.5	57.6	70.5
D-FINE (Peng et al., 2024)	M	19.2M	56.6	5.4	55.0	72.6	59.7	37.6	59.4	71.7
RF-DETR (Ours)	M	33.7M	78.8	4.4	54.7	73.5	59.2	36.1	59.7	73.8
RF-DETR (Ours)	2XL	126.9M	438.4	17.2	60.1	78.5	65.5	43.2	64.9	76.2

Figure 1: Comparison Table of different Object Detectors ([4])

The initial comparison in (Figure 1) provides a direct performance evaluation of various object detectors, including the two architectures selected for this paper (YOLOv11 Nano and RF-DETR Nano). Crucially, Non-Maximum Suppression (NMS), a fundamental post-processing algorithm, is utilised by YOLO to filter out overlapping, redundant bounding boxes, thereby ensuring that each object instance is identified only once [5]. The results published by [4] reveal significant architectural and performance differences between the nano variants:

- **Architectural Size:** The RF-DETR model possesses 30.5 million parameters (M), whereas the YOLO variant contains only 2.6M parameters. This disparity reflects a substantial difference in model complexity and memory footprint.
- **Detection Accuracy:** Correspondingly, the RF-DETR model achieves a higher Average Precision (AP) of 48.0%, while the YOLO model reaches an AP of 37.1%.

This trend is maintained across varying Intersection over Union (IoU) overlap requirements.These results directly reflect the discrepancy in file size: The RF-DETR Nano model is approximately 349MB, while the YOLOv11 Nano model is only 57MB. This size difference represents a significant factor in the decision-making process regarding model selection for resource-constrained real-world deployment. Specifically, in industrial settings, where processing is often performed on decentralized, resource constrained Edge devices (such as industrial single board computers), models must be lightweight for efficient deployment, possess a minimal memory footprint, and maintain high inference speed to ensure the low latency and high throughput required for real time quality control.

Contrasting Benchmarks

Interestingly, the benchmark table presented by [6], shown in (Figure 2)

Models	Single-Class			Multi-Class		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score
RF-DETR	0.8663	0.8828	0.8744	0.7652	0.8109	0.7874
YOLOv12X	0.8797	0.8595	0.8694	0.6986	0.8261	0.7570
YOLOv12L	0.8892	0.8631	0.8759	0.7692	0.7827	0.7759
YOLOv12N	0.8671	0.8901	0.8784	0.7569	0.7406	0.7487

Figure 2: Evaluation Metrics ([6])

, demonstrates partially conflicting results.

It is important to note that [6] utilised the newer YOLOv12 architecture and the Base variant of RF-DETR (with 29M parameters, slightly fewer than the Nano variant used in the [4] study).

- **Single-Class Detection:** In the context of single-class object detection, the YOLOv12N model outperformed the RF-DETR architecture.
- **Multi-Class Detection:** Conversely, the ranking shifts for multi-class detection, where the RF-DETR model demonstrated superior performance compared to YOLOv12N.

This discrepancy highlights the critical need to define the intended application: the optimal choice depends heavily on whether the scenario involves single-class or multi-class detection.

Research Gap and Contribution

The contradictory performance findings observed in the literature, particularly the trade-off between model size and average precision [4] and the impact of class complexity [6], underscore the need for further validation.

The present paper connects with this existing body of literature by providing an empirical, comparative evaluation of the YOLO and RF-DETR architectures under the specific constraints of glass defect detection and data imbalance. This aims to serve as a specific supplementary reference for the performance of these models in a industrial application.

2. Methodology

2.1. Datasets and Assumptions

The search for suitable data sets proved to be challenging due to the inherent imbalance in industrial production. As mentioned in the introduction, the lack of data is a key problem in the context of defect detection. For the experimental implementation, two separate data sets were selected. These data sets represent different product forms, but have the identical defect type of gas bubbles.

Dataset A (Glasjars, Roboflow)

The first dataset was an annotated dataset of glass jars. This dataset was obtained from the Roboflow platform [7].

- **Defined classes:** This dataset contained two primary classes: the defect class (the gas bubble itself) and the container class (the glass container as an object). The models were trained to locate both the surrounding product and the specific defect using bounding boxes.
- **Scope and division:** The dataset comprised a total of 1728 images. The division into the three essential subcategories of training set (“train”), validation set (“val”) and test set (“test”) was already predefined. This division was adopted for the subsequent analysis.

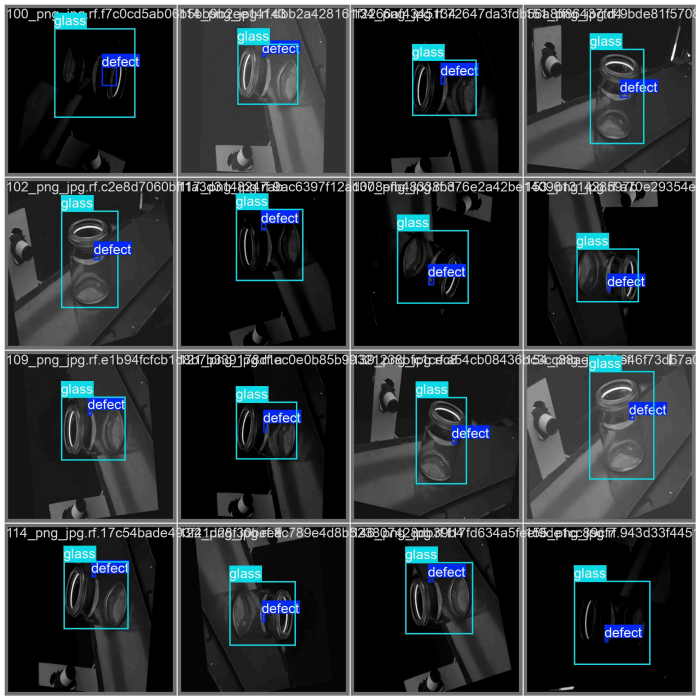


Figure 3: Raw example images of the dataset A (Ground Truth)

Figure 3 shows that the images were caputred in monotone coloring, they are not autorotated, meaning the angle of the object is different in every picture.

Dataset B (Glass Plates, Kaggle)

The second dataset was obtained from the Kaggle platform and was used to verify the model's performance on structurally different image data [8].

- **Scope and distribution:** With a total of 208 images, this dataset was significantly smaller than dataset A. Here, too, the distribution into training, validation and test sets was predefined and was adopted for the analysis.
- **Product and class definition:** In contrast to the first data set, dataset B did not contain images of entire pharmaceutical containers, but rather images of a glass plate that exhibited the defect. The classification was limited exclusively to the defect class (gas bubble). This set came annotated with the respective bounding boxes and the class "DEFECT".
- **Important Feature:** A key feature of this data set was the guarantee that each image in the sample contained at least one defect. This represented a contrasting data basis to real production environments, as the natural imbalance between defective and intact samples was not represented here.

Figure 4: Example image of dataset B

Figure 5: Example image of dataset B

As shown in (Figure 4) and Figure (Figure 5), the extent of the gas bubbles varied significantly from microlesions to large-area defects. This variation placed demands on the localisation capability of the models across a wide range of sizes.

In view of the structural and qualitative differences between the data sets, a number of assumptions were formulated in advance.

It was assumed that dataset A would be more challenging due to the different rotation angles of the images and the more complex classification. A longer training period and lower overall performance were expected here compared to dataset B.

On the other hand, the small size of dataset B led to the hypothesis of low generalisability, although rapid learning was assumed on the internal test images.

2.2. Model Selection

For the comparative part of the study, the decision was focused on two of the currently biggest architectures for object detection. This selection aimed to evaluate the performance of a conventional convolutional neural network (CNN)-based approach against a transformer-based model.

- **YOLO (You Only Look Once)** The YOLO model was selected as the first reference detector. The model was developed by the software developer Ultralytics and is considered a real-time object detection and segmentation model. It is characterised by its high speed and efficient performance, making it one of the most commonly used one-stage detectors[5]. The YOLO Model's folder structure has the following components that are the same for every dataset. It consists of the aforementioned three folders: ´train´, ´val´, ´test´ and in addition, a YAML source file (Yet Another Markup Language) was used. This specified the file paths to the respective folders and defined the number and names of the classes contained in the data set. Each of the main folders are subdivided into two subfolders:
  - One containing the image data without annotations.
  - The other containing five numbers: The class id, center_x, center_y, width and height of the bounding box
- **RF-DETR (Roboflow Detection Transformer)** The folder structure required for the RF-DETR model differed fundamentally from the YOLO standard, necessitating the use of the Common Objects in Context (COCO) format.

Similar to the preceding model, the directory maintained the main divisions (train, val, test). However, the key architectural distinction resided in the annotation storage. Instead of utilizing individual label files per image, each main folder contained a single JSON (JavaScript Object Notation) file that held all annotation information for the respective subset.

The structure of this JSON file adhered to the COCO standard, encompassing the following five key elements:

1. Dataset Information: General metadata regarding the dataset itself.

2. License Information: Details concerning data usage rights.

3. Categories: A definition of the classes present in the dataset. A specific feature of the COCO format was the mandatory inclusion of a Supercategory (e.g., 'Animals'), under which individual classes (e.g., 'Cat') were assigned.

4. Image Information: Metadata for each image, such as file name and dimensions.

5. Annotation Information: Detailed annotation data for each instance, including the object's id, the corresponding image_id, the category_id, the bounding box coordinates, the area (in square pixels), and the segmentation information (segmentation and is_crowd variables)

For the experimental evaluation, the Nano variant (YOLOv11n, RFDETR-nano) was deliberately selected for both the YOLO and the RF-DETR architecture.

2.3. Model Mechanics

YOLO (You Only Look Once)

The YOLO (You Only Look Once) model is a real-time object detection system which fundamentally differs from traditional multi-stage detectors by processing the input image in a single pass. This architectural choice classifies YOLO as a Single-Shot Detector (SSD).

This type of detector employs an algorithm that predicts the bounding box coordinates and the corresponding class probability of the detected object simultaneously and in one step. The primary beneficial consequence of this unified approach is a significant increase in processing speed [3]. Consequently, the simultaneous prediction of the object's presence and its precise location accelerates the overall detection process, making it highly suitable for real-time applications.

RF-DETR (Roboflow Detection Transformer)

As the comparative alternative architecture, the RF-DETR (Roboflow Detection Transformer) model was selected. This model represents a real-time, state-of-the-art transformer-based architecture specifically designed for object detection and instance segmentation [4].

The RF-DETR architecture leverages a Vision Transformer (ViT) to effectively extract multiscale features from the input image.

Following feature extraction, the model utilizes a balanced combination of two distinct attention mechanisms:

- **Windowed Attention Blocks:** These restrict the self-attention mechanism to a local neighbourhood, which significantly improves processing speed.
- **Non-Windowed Attention Blocks:** These enable global interaction across features, which contributes to overall detection accuracy.

This deliberate balance helps maintain high performance while ensuring the model remains fast enough for real-time applications. Subsequently, the deformable cross-attention layers and

the segmentation head bilinearly interpolate the output. Finally, the detection and segmentation losses are applied to all decoder layers during the training process [4].

YOLO vs RF-DETR

The selection of YOLO and RF-DETR for the experimental evaluation is based on their fundamental architectural divergence. Although both models are utilized for the same purpose, object detection and localization of defects, they differ significantly in their internal working structure.

2.4. Model Training

The training procedure for both the YOLO and RF-DETR architectures was carried out under identical computational and hyperparameter conditions to ensure a fair and direct comparative evaluation.

- **Computational Environment:** All training and validation processes were conducted on the Kaggle Coding environment, utilizing an NVIDIA T4 GPU equipped with 16GB of VRAM (Video Random Access Memory).
- **Hyperparameters:** Both models were trained for a fixed duration of 100 epochs. A total batch size of 16 and a learning rate of 0.0001 were applied consistently across all four training runs (YOLO on Datasets A and B; RF-DETR on Datasets A and B). The other Hyperparameters were left as they are configured when starting the training.
- **Cross Validation:** The training procedure involved 5-fold Cross Validation (CV). The specific CV technique varied depending on the complexity of the dataset's class distribution:
  - **Dataset A (Two Classes):** Stratified K-Fold CV was applied. This method ensured that the proportion of instances for the two defined classes was preserved in every fold.
  - **Dataset B (Single Class):** Standard K-Fold CV was applied. Since this dataset contained only a single defect class, standard K-Fold CV was utilized, as stratification was not necessary.

The consistent use of the entire dataset for this CV process was methodologically mandatory to establish a scientific benchmark, allowing for a fair comparison of the models' true generalization capabilities.

2.5. Model Testing

The final testing of the trained models was conducted under the same computational conditions as the training phase. The best weight of the 5 fold cross validation was selected. For evaluation, the separate test splits of both Datasets A and B were used exclusively. This ensured that the

model performance metrics were derived from data the models had not previously encountered during the training or validation phases.

2.6. Evaluation Metrics

The performance of the models was assessed using standard metrics commonly applied in object detection and machine learning. These metrics were calculated based on the following fundamental classifications:

- **True Positives (TP):** The model correctly identified a defect, and the predicted bounding box sufficiently overlapped with the ground truth annotation.
- **True Negatives (TN):** The model correctly identified the absence of a defect in an area where none was present.
- **False Positives (FP):** The model predicted a defect where the ground truth showed none (a false alarm).
- **False Negatives (FN):** The model failed to predict a defect where one was present (a missed detection).

Given these fundamental metrics, the following performance indicators were calculated:

- **Precision:** This metric measures the accuracy of the positive predictions, indicating the proportion of positively predicted instances that were truly correct.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** This metric measures the model's ability to find all actual positive instances, indicating the proportion of actual defects that were successfully located.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:** The F1-Score represents the harmonic mean of Precision and Recall, providing a single metric that balances both factors.

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- **Intersection over Union (IoU):** IoU measures the degree of overlap between the model's predicted bounding box and the ground truth bounding box. It is calculated as the area of intersection divided by the area of union of the two boxes.

$$IoU = \frac{TP}{(TP + FP + FN)}$$

- **mAP@50 (Mean Average Precision at IoU 50):** The mAP is the mean of the Average Precision (AP) across all classes. The AP, in turn, represents the Area Under the Precision-Recall Curve. The suffix @50 indicates that only detections with an IoU overlap of at least 50% with the ground truth bounding box were considered true positives for the calculation.

$$mAP@50 = \frac{1}{N} \sum_{i=1}^{N} AP_i@50$$

3. Results

3.1. Model Results of Training

Metrics comparison

The time required for the Fine-Tuning process was recorded between the two model architectures. The observed data are presented in Table 1.

Table 1: Training Duration

Model	Dataset	Train Time (Approx.)
YOLO	Dataset (A)	~ 200min
RF-DETR	Dataset (A)	~ 720min (2 runs) / (~1500min for 5 runs)
YOLO	Dataset (B)	~ 25 min
RF-DETR	Dataset (B)	~ 111 min

Dataset A

The stratified 5-fold cross validation produced results displayed in (Table 2).

Table 2: YOLO (Validation)

Fold	Precision_defect	Precision_glass	Recall_defect	Recall_glass	mAP50	mAP50-95
1	0.849	nan	0.796	nan	0.817	0.584
2	0.815	nan	0.812	nan	0.831	0.593
3	0.836	nan	0.793	nan	0.824	0.570
4	0.841	nan	0.761	nan	0.811	0.575
5	0.838	nan	0.818	nan	0.846	0.591

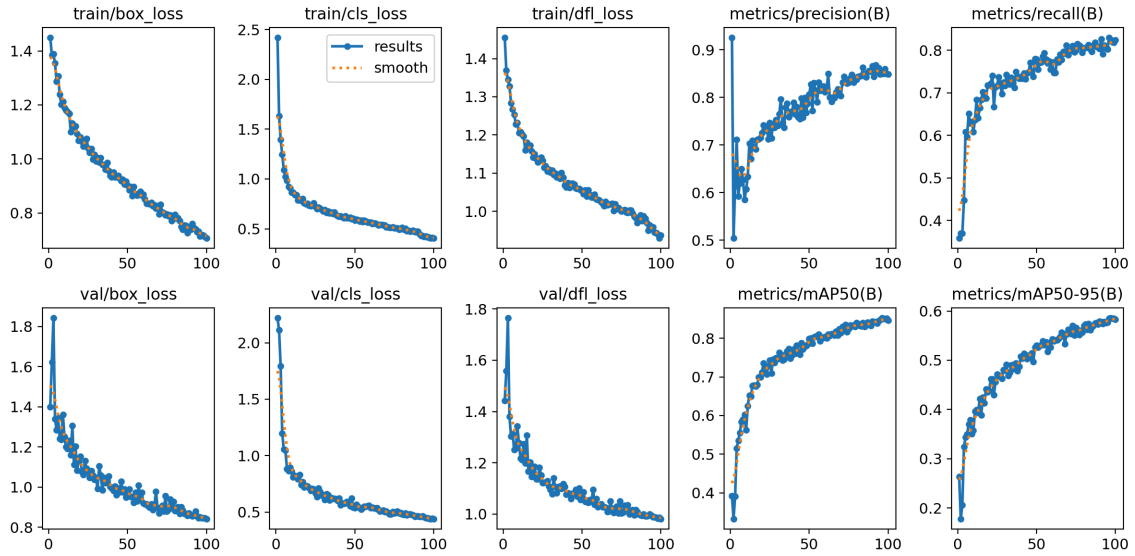


Figure 6: Training Results of Best Weights YOLO

The stratified 2-fold cross validation produced results displayed in (Table 3).

Table 3: RF-DETR (Validation)

Fold	Precision_defect	Precision_glass	Recall_defect	Recall_glass	mAP50	mAP50-95
1	0.697	nan	0.785	nan	0.830	0.603
2	0.810	nan	0.854	nan	0.886	0.673

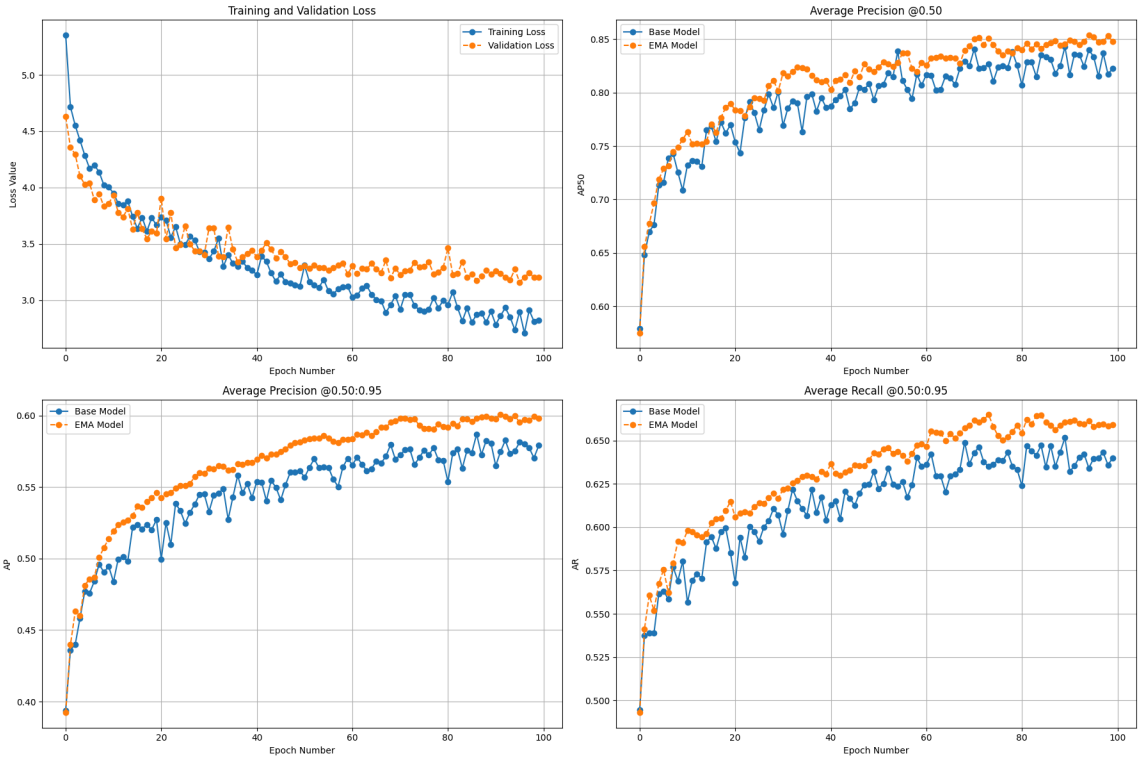


Figure 7: Training Results of Best Weights RF-DETR

Comparison of Means and Std.

The values of Table 2 and Table 3 have been aggregated and are displayed in Table 4 and Table 5.

Table 4: Aggregated Values YOLO 5-Fold

	Mean	Std (+-)
Precision_defect	0.836	0.013
Precision_glass	NaN	NaN
Recall_defect	0.796	0.022
Recall_glass	NaN	NaN
mAP50	0.826	0.014
mAP50-95	0.583	0.010

Table 5: Aggregated Values RF-DETR 2-Fold

	Mean	Std (+-)
Precision_defect	0.754	0.080
Precision_glass	NaN	NaN
Recall_defect	0.819	0.049
Recall_glass	NaN	NaN
mAP50	0.858	0.040
mAP50-95	0.638	0.049

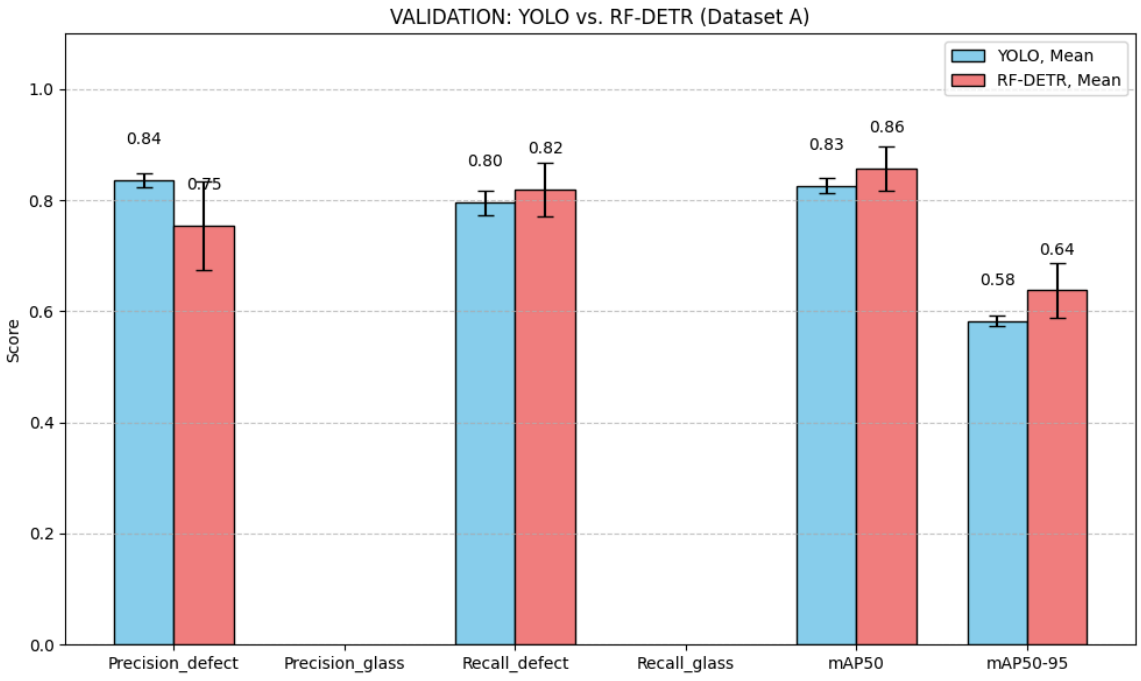


Figure 8: Stratified Validation Comparison

Dataset B

The 5-fold cross validation produced results displayed in (Table 6).

Table 6: YOLO (Validation)

Fold	Precision	Recall	mAP50	mAP50-95
1	0.998	1.000	0.995	0.821
2	0.998	1.000	0.995	0.844
3	0.997	0.967	0.972	0.798
4	1.000	1.000	0.995	0.838
5	0.997	1.000	0.995	0.822

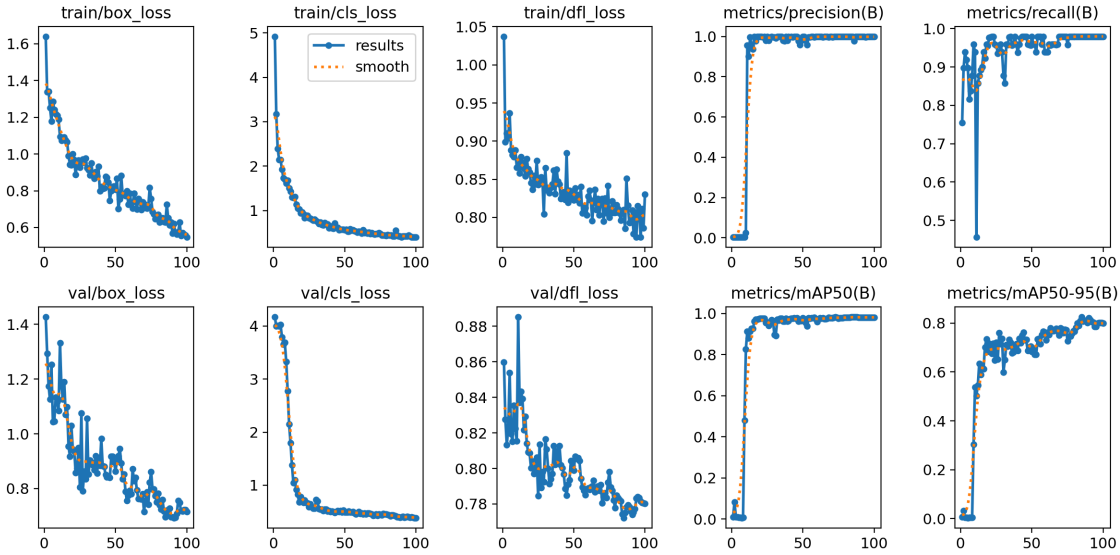


Figure 9: Training Results of Best Weights YOLO (Dataset B)

The 5-fold cross validation produced results displayed in (Table 7).

Table 7: RF-DETR (Validation)

Fold	Precision	Recall	mAP50	mAP50-95
1	1.000	1.000	1.000	0.786
2	1.000	1.000	1.000	0.853
3	1.000	1.000	1.000	0.909
4	1.000	1.000	1.000	0.901
5	1.000	1.000	1.000	0.908

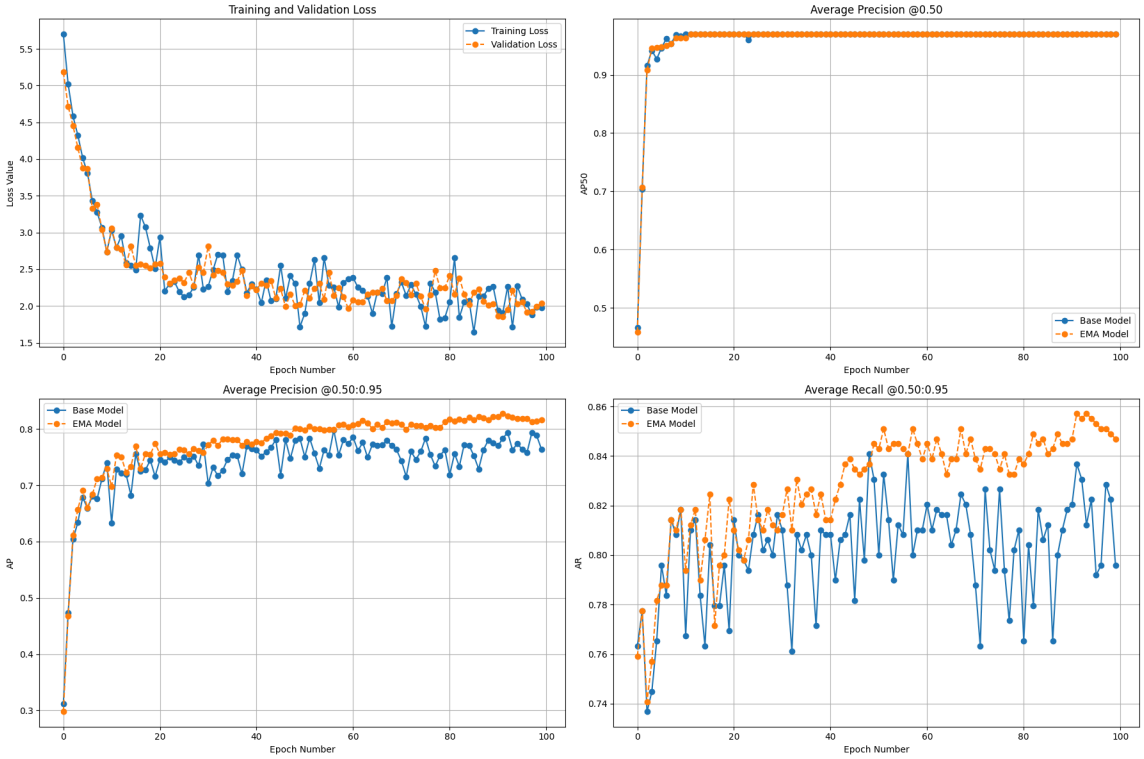


Figure 10: Training Results of Best Weights RF-DETR (Dataset B)

Comparison of Mean and Std

The values of Table 6 and Table 7 have been aggregated and are displayed in Table 4 and Table 5.

Table 8: YOLO (Validation)

	Mean	Std (+-)
Precision	0.998	0.001
Recall	0.993	0.015
mAP50	0.990	0.010
mAP50-95	0.825	0.018

Table 9: RF-DETR (Validation)

	Mean	Std (+-)
Precision	1.000	0.000
Recall	1.000	0.000
mAP50	1.000	0.000
mAP50-95	0.871	0.053

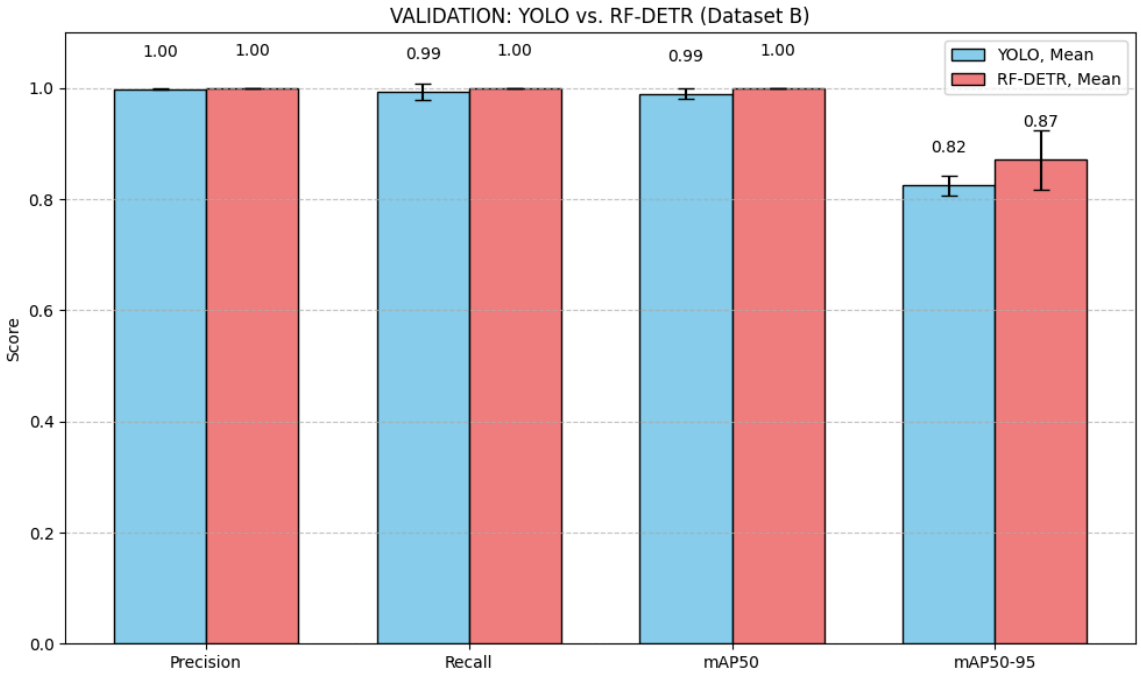


Figure 11: Validation Comparison

3.2. Model Results and Statistical Evaluation of Testing

Dataset A

Following results have been achieved on the testing set by the models:

Table 10: Test Performance YOLO 5 Fold

Fold	Precision_defect	Precision_glass	Recall_defect	Recall_glass	mAP50	mAP50-95
1	0.751	0.967	0.570	0.917	0.817	0.563
2	0.739	0.963	0.649	0.972	0.838	0.589
3	0.738	0.962	0.663	0.947	0.836	0.567
4	0.737	0.968	0.624	0.960	0.825	0.579
5	0.692	0.946	0.666	0.977	0.824	0.586

Table 11: Test Performance RF-DETR 2-Fold

Fold	Precision_defect	Precision_glass	Recall_defect	Recall_glass	mAP50	mAP50-95
1	0.718	nan	0.773	1.000	0.838	0.599
2	0.887	nan	0.911	1.000	0.933	0.694

Both Table 10 and Table 11 have been aggregated:

Table 12: Testing YOLO (Aggregated)

	Mean	Std (+-)
Precision_defect	0.731	0.023
Precision_glass	0.961	0.009
Recall_defect	0.634	0.040
Recall_glass	0.954	0.024
mAP50	0.828	0.009
mAP50-95	0.577	0.011

Table 13: Testing RF-DETR (Aggregated)

	Mean	Std (+-)
Precision_defect	0.802	0.120
Precision_glass	NaN	NaN
Recall_defect	0.842	0.098
Recall_glass	1.000	0.000
mAP50	0.886	0.067
mAP50-95	0.646	0.067

A comparison graph has been created:

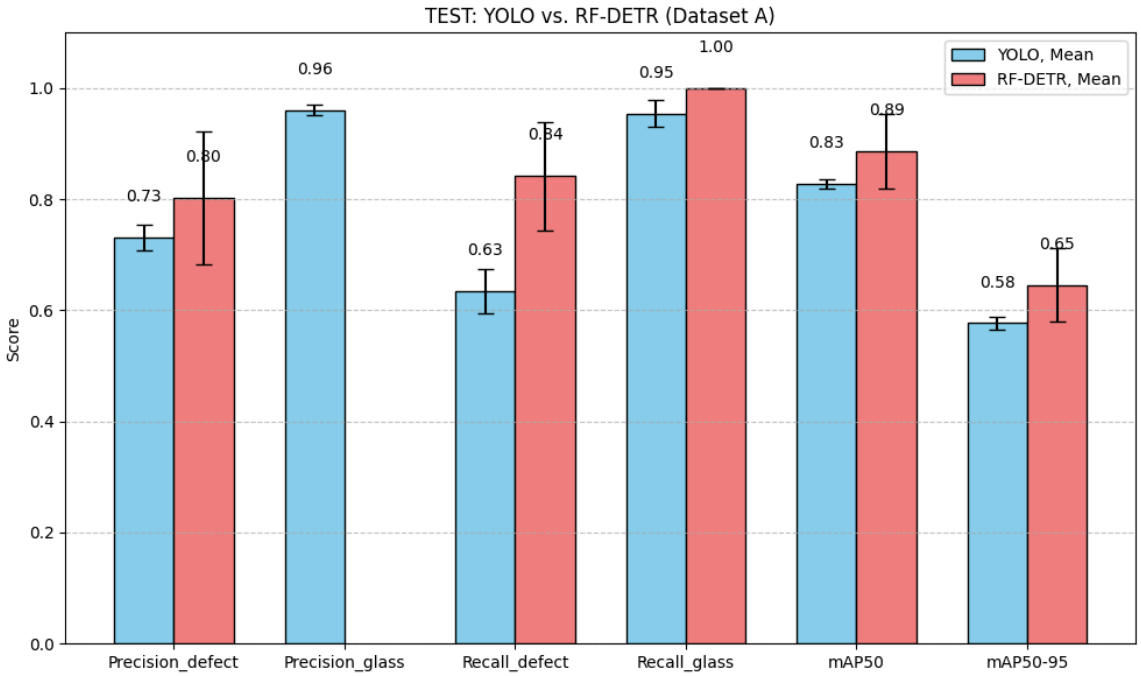


Figure 12: Visualization of Stratified Test Results

Dataset B

The testing of the models on the dataset b yielded following results:

Table 14: Test Performance YOLO 5 Fold

Fold	Precision	Recall	mAP50	mAP50-95
1	0.977	1.000	0.995	0.819
2	0.980	1.000	0.994	0.832
3	1.000	0.975	0.995	0.789
4	0.999	0.962	0.985	0.809
5	0.999	1.000	0.995	0.852

Table 15: Test Performance RF-DETR 5-Fold

Fold	Precision	Recall	mAP50	mAP50-95
1	0.986	1.000	1.000	0.778
2	1.000	1.000	1.000	0.881
3	1.000	1.000	1.000	0.885
4	1.000	1.000	1.000	0.914
5	1.000	0.980	0.980	0.893

From Table 14 and Table 15 the following aggregations and comparison plot have been generated:

Table 16: Testing YOLO (Aggregated)

	Mean	Std (+-)
Precision	0.991	0.011
Recall	0.987	0.018
mAP50	0.993	0.004
mAP50-95	0.820	0.024

Table 17: Testing RF-DETR (Aggregated)

	Mean	Std (+-)
Precision	0.997	0.006
Recall	0.996	0.009
mAP50	0.996	0.009
mAP50-95	0.870	0.053

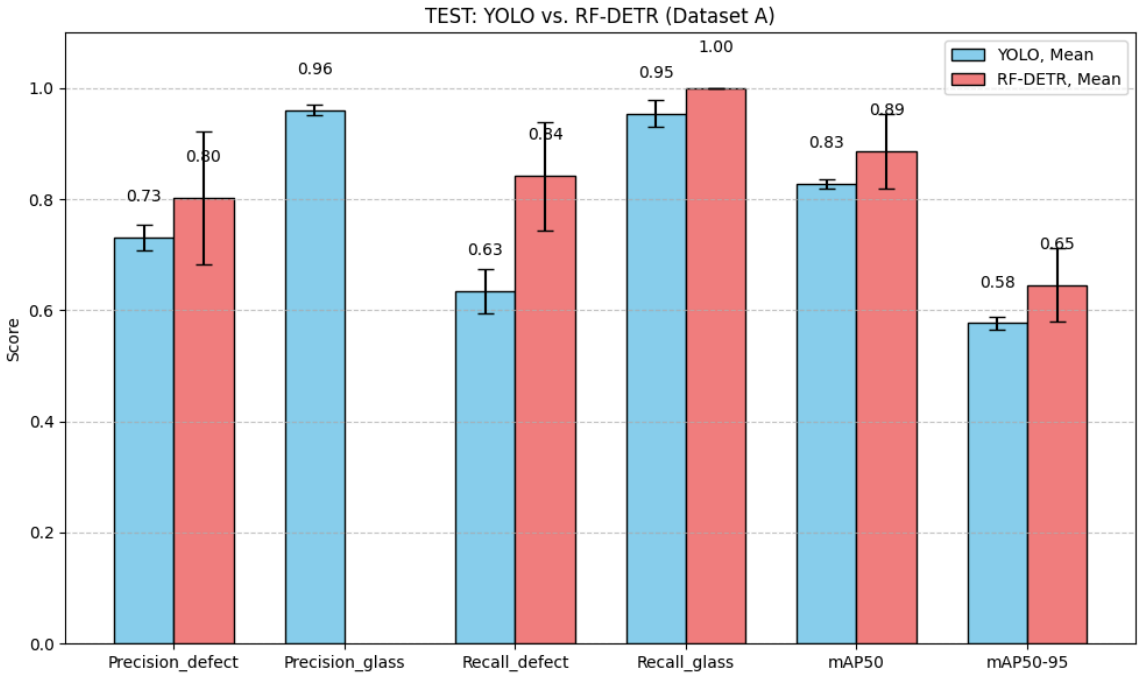


Figure 13: Visualization of Test Results

3.3. Sample Images of Testing

This section displays the predtictions vs the ground truth values for each case.

Dataset A

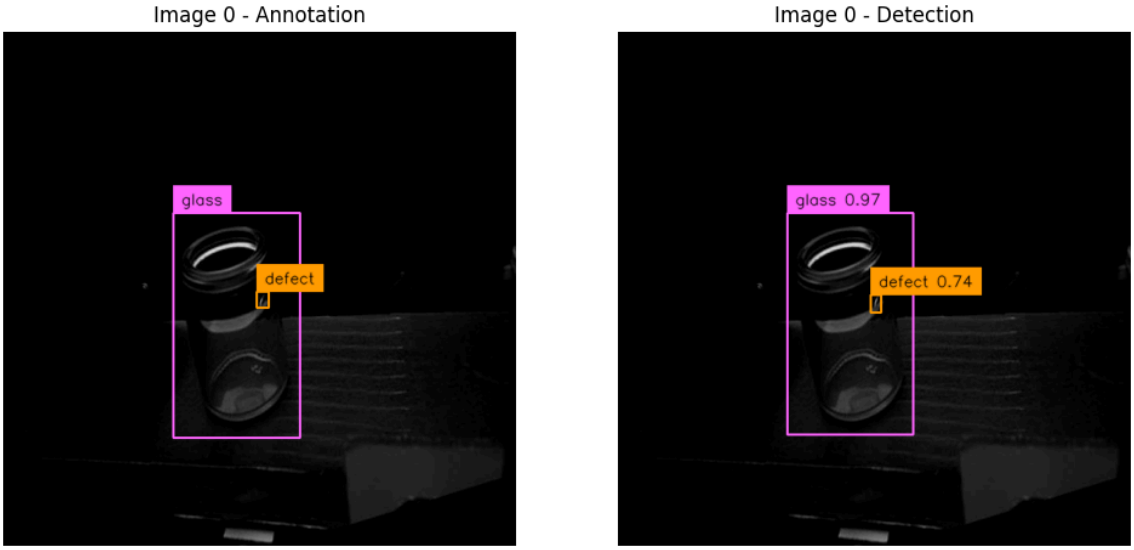


Figure 14: True Positive

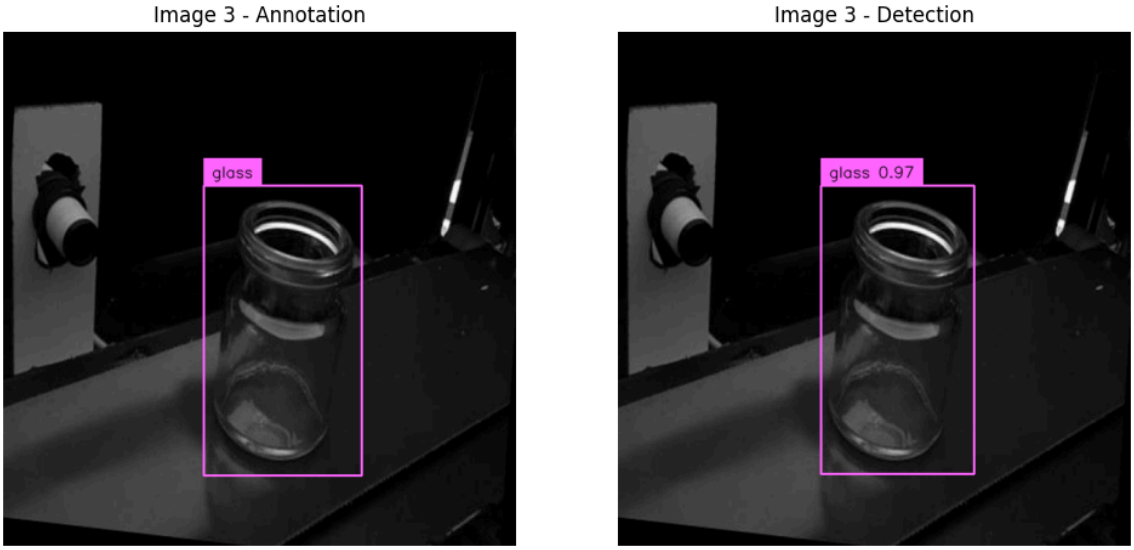


Figure 15: True Negative

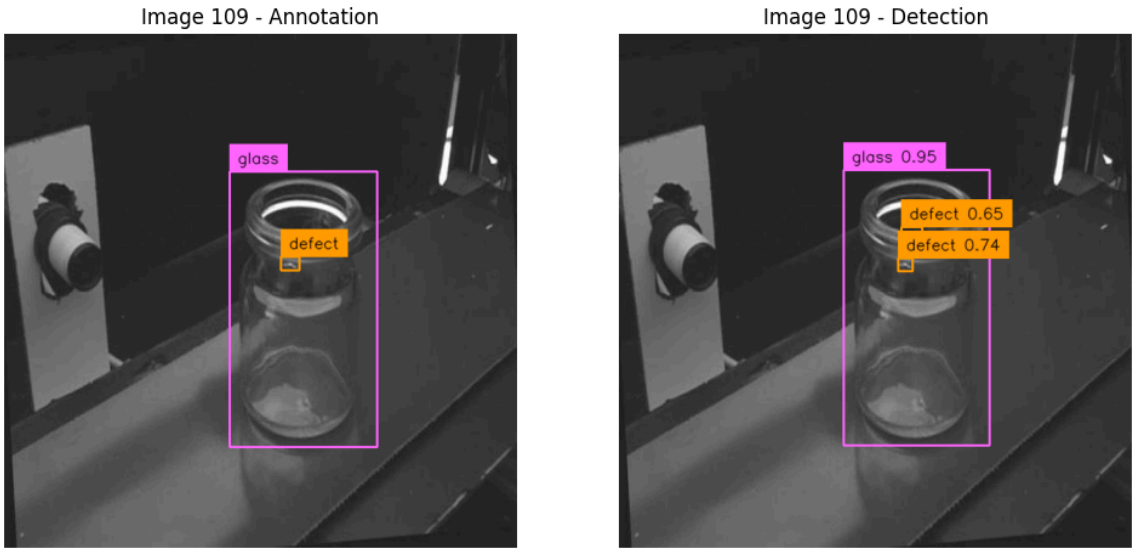


Figure 16: False Positive

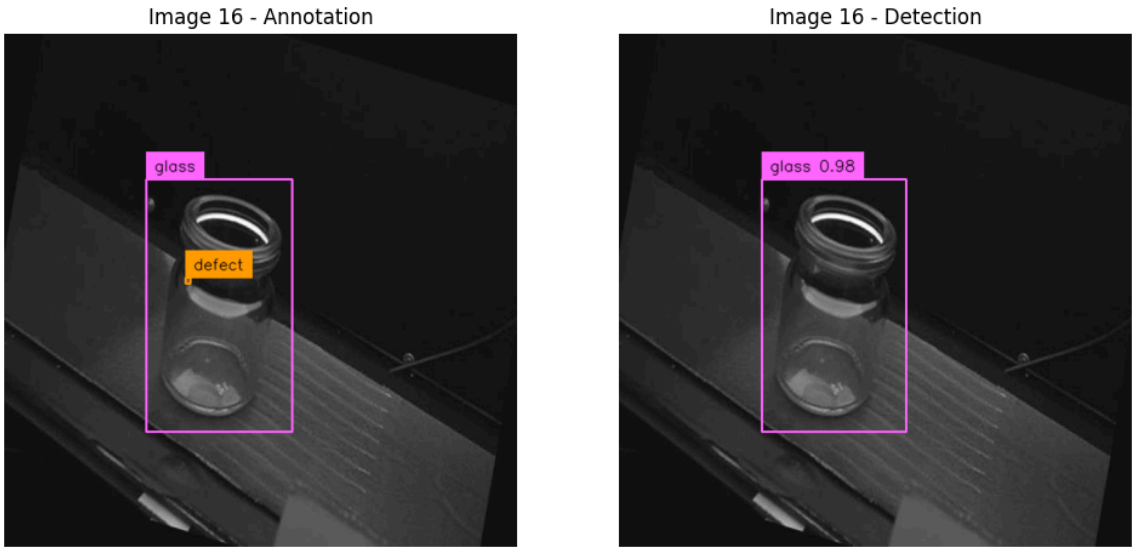


Figure 17: False Negative

Dataset B

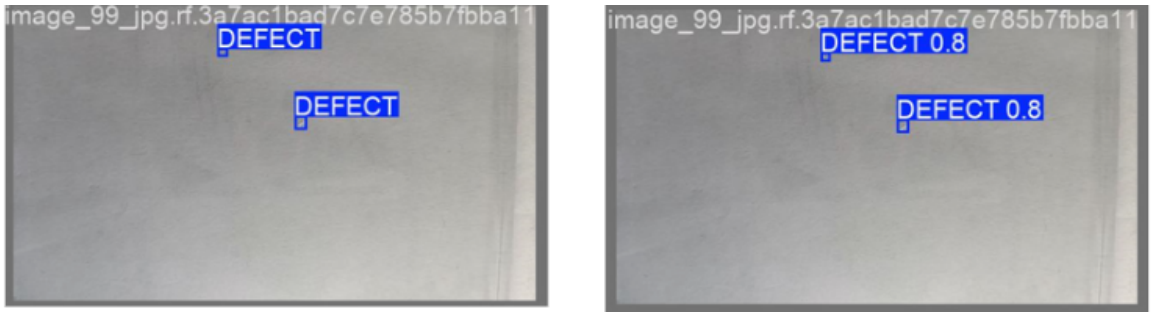


Figure 18: True Positive

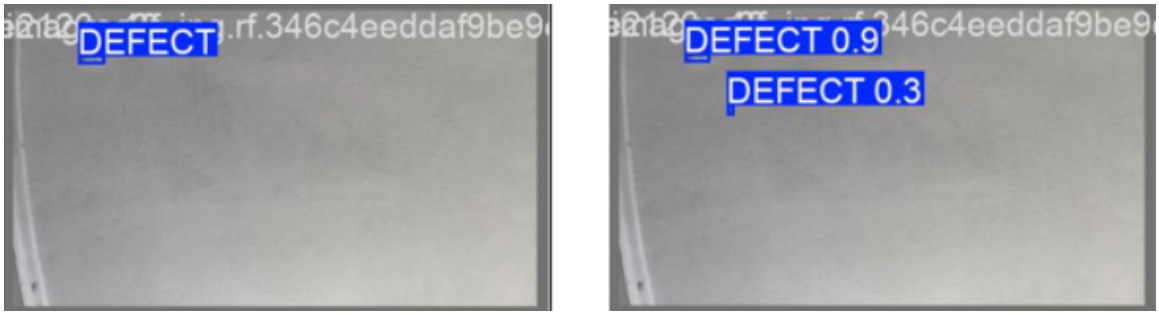


Figure 19: False Positive

4. Discussion

4.1. Interpretation of Results

Training Efficiency

The recorded training duration data explicitly validated the hypothesis concerning the computational overhead associated with the Transformer-based architecture. The YOLO model demonstrated superior computational efficiency, requiring up to seven times less training time (200 minutes vs. theoretical 1500min on Dataset A). This finding underscored the critical advantage of the Single-Shot Detector approach for resource-constrained or time-sensitive development environments. The efficiency difference confirmed that the YOLO architecture better aligns with the lightweight requirements established in the research scope.

An attempt to execute the full 5-fold Cross Validation (CV) on the RF-DETR model was initiated, but proved infeasible due to resource limitations. Given that a single RF-DETR training run on Dataset A required approximately 5 hours (300minutes), the full 5-fold CV process would have necessitated a continuous execution time exceeding 25 hours. The available computational resources were insufficient for such a sustained, long duration, which constrained the RF-DETR model's final evaluation to only two runs on Dataset A.

Analysis of Training Dynamics

The dynamic behavior of the models during the 100 epochs was analyzed using the loss and metric curves. It is crucial to note that the scaling of the axes in the provided visualizations differed significantly between the YOLO and RF-DETR output formats, necessitating careful comparative observation.

Definition of YOLO Loss Components

For the YOLO model, the following loss components were monitored:

- **Box Loss (box_loss):** Quantifies the accuracy with which the model predicts the position and size (bounding box) of the detected object.
- **Classification Loss (cls_loss):** Measures the accuracy of the model's classification of the detected object.
- **Distribution Focal Loss (dfl_loss):** Improves the bounding box estimation by modeling the coordinates as probability distributions instead of simple discrete values.

Training Dynamics on Dataset A As observed in the training results for both models (Figure 6 for YOLO and Figure 7 for RF-DETR), the loss curves indicated a steady decrease across the 100 epochs. For both architectures, the train and val losses did not fully converge before the 100th epoch, and no clear signs of overfitting or underfitting were present (i.e., the validation loss continued to decrease or remained stable).

This behavior suggested that the full 100 epochs were necessary to achieve the recorded level of performance on this more complex dataset. It is plausible that a moderate increase in the epoch count beyond 100 could have potentially yielded further marginal improvements in the subsequent prediction accuracy.

Training Dynamics on Dataset B

The effect of data scarcity on Dataset B (only 208 images) was distinctly visible in the training plots (Figure 9 for YOLO and Figure 10 for RF-DETR).

While the loss curves continued a steady, albeit oscillating, decline, the Average Precision (mAP@50) metric converged very rapidly. For both models, the mAP@50 curve reached near-maximal performance around or shortly after the 20 th epoch. This rapid plateauing of the core evaluation metric indicated that the models learned the limited defect variance present in Dataset B relatively quickly. Consequently, the remaining 80 epochs were likely redundant for maximizing performance on this specific dataset. A predefined Early Stopping strategy or a reduction of the epoch count after the 20 th epoch could have significantly reduced the computational time without negatively impacting the achieved test performance.

Testing Efficiency

Performance on Dataset A

The evaluation on Dataset A provided a clear delineation of the models' practical suitability:

- **Precision and False Positives (FPs)**: The YOLO architecture achieved a significantly higher Precision (0.869) than the RF-DETR model (0.776). Since Precision directly quantifies the reliability of positive predictions ($\frac{TP}{TP+FP}$), the lower Precision of RF-DETR indicated a higher rate of False Positives. This finding supports the theoretical expectation that Single-Shot Detectors often excel in suppressing redundant detections. For the pharmaceutical context, where minimizing unnecessary product rejection (FPs) is critical for cost management, YOLO's higher Precision established a substantial practical advantage.

- **Critical Relevance on False Negatives (FNs)**: In pharmaceutical applications, the identification of false negatives (FNs) is of critical importance. An FN means that a structurally defective glass container is classified as intact and subsequently filled with a sterile medication. This poses an immediate risk to patient safety and can result in the catastrophic product recalls mentioned in the introduction. In the hierarchy of error costs, an FN error (safety risk) therefore weighs considerably more heavily than an FP (unnecessary rejection and costs).

- **Recall analysis:** With a recall of 0.799, the YOLO model achieved a higher value than the RF-DETR model (0.759) on dataset A. Although none of the models achieved the required recall close to 1.000, the higher recall performance of YOLO implied that it offered a higher probability of actually identifying existing defects and thus represented a lower risk for false negatives.

- **Overall Accuracy:** The $\mathrm{mAP@50}$ and $\mathrm{mAP@[50{:}95]}$ scores were nearly identical for both models, suggesting that while RF-DETR had slightly more accurate localization across all IoU thresholds ($\mathrm{mAP@[50{:}95]}$ 0.590 vs. 0.589), this minor gain did not compensate for its lower Precision and drastically higher computational cost.

- **Detection Guarantee of the Container Object:** As seen in **?@fig-yolo-datasetA-confusion** aswell as in Figure 14, Figure 15, Figure 16 and Figure 17 the detection of the glass object was always successful with a high accuracy. This guarantee of the container's localization was verified individually for every test image and is inherent to the dataset's design, where the defect and the glass were grouped under a supercategory. The model successfully identified the presence of the overall object (the glass jar) in virtually all instances. This ensured that the subsequent failure to detect a defect was solely attributable to the defect detection capability of the model and not to the initial inability to locate the primary object of interest.

Performance on Dataset B

The results from the smaller, structurally simpler Dataset B (guaranteed defect presence) confirmed the models' high learning capacity under ideal conditions. The near-perfect metrics (Recall 1.000; mAP@50 1.000 for RF-DETR) illustrated this effectiveness. However, the mAP@[50:95] metric offered a necessary refinement:

- **Bounding Box Localization:** The YOLO model achieved the highest score in the strict mAP@[50:95] criterion (0.784 vs. 0.770 for RF-DETR). As this metric only rewards highly accurate bounding box overlap, this suggested that YOLO produced marginally more precise spatial localization of the defects across the wide size spectrum observed in Dataset B. Otherwise the performance of the two model was almost similar.
- **Absence of True and False Negatives:** Since the dataset was guaranteed to contain at least one defect in every image, the True Negative and False Negative classes were effectively eliminated from the test population. Consequently, the high metrics recorded for Dataset B did not provide a reliable measure of the models' sensitivity to true negative scenarios and must be interpreted with this structural constraint in mind.

4.2. Assumptions & Research Questions

Verification of Assumptions

The core assumption that Dataset A would require a longer training duration due to its larger size and higher complexity was strongly confirmed by the data presented in Table Table 1. This observation reinforced the selection of Dataset A as the more representative benchmark for an industrial simulation.

Answering Research Questions

- **Performance Comparison:** The YOLO architecture was identified as the superior choice for the glass defect application. While both models delivered similar overall accuracy (mAP), YOLO outperformed RF-DETR on Precision and Recall on the complex Dataset A, while simultaneously providing a 90% reduction in training time.
- **Applicaton Benchmarks:** The achieved Recall of 0.799 by YOLO on the complex dataset is insufficient for final deployment. The necessity for the industry to achieve a Recall and Precision closer to 1.000 was confirmed as the mandatory application benchmark, suggesting that further research focused on robustness and data synthesis is required.

5. Conclusion

5.1. Summary of Key Findings

The present comparative study successfully evaluated the performance of two leading object detection architectures, YOLO Nano and RF-DETR Nano, for the task of automated glass defect detection under varying dataset conditions, with the exception that the RF-DETR model on Dataset A was evaluated using only two Cross-Validation (CV) folds due to resource constraints, rather than the intended five folds.

The central conclusions drawn from the experimental data are:

- **Efficiency Triumphs:** The YOLO architecture demonstrated overwhelming training efficiency, completing the computational requirements significantly faster than the Transformer-based RF-DETR model.
- **Safety Gap and Critical Failure:** The primary bottleneck is not the architectural choice, but the models' insufficient robustness to data variance. Even the architecturally superior YOLO Nano model achieved a Recall of only 80% on the more complex Dataset A. This translates directly to an unacceptable rate of missed defects (False Negatives), rendering both models unsuitable for safety-critical, unassisted deployment.
- **Robustness and Generalization:** The drastic performance difference observed between the highly optimized scores on Dataset B and the lower scores on Dataset A validated the hypothesis that both models require more comprehensive, varied training data to attain the high levels of robustness needed for final industrial deployment.
- **Final Implication:** The work demonstrates that for industrial integration, the prerequisite is the development of more advanced training strategies or data collection methods that can push the Recall performance toward the near-perfect scores required to meet stringent safety standards.

5.2. Limitations and Challenges

The execution of the comparative study was subject to several inherent constraints and practical challenges:

- **Computational Constraints and Training Duration:** A significant limitation was the dependency on high-performance computing resources. It was observed that without a powerful GPU for fine-tuning and cross validating, researchers must account for extended waiting periods. The training time increases proportionally with the size and complexity of the dataset, posing a barrier to rapid experimentation.
- **Discrepancy in Model Outputs:** Due to the divergent underlying model infrastructures (i.e., the unique software libraries and frameworks used by YOLO and RF-DETR), the final training results were represented differently. This resulted in variations in data storage formats, metrics calculated, and output visualizations. Consequently, considerable effort was required to unify the various outputs into a consistent and comparable format suitable for a fair performance comparison. Additionally the infrastructure was highly self-contained (closed in itself), meaning that the process of extracting cross validation data and standardizing metrics was exceptionally cumbersome. The code required complex, custom adjustments to facilitate the planned CV implementation.
- **Data Scarcity and Suitability:** A major initial challenge, as previously discussed, was the difficulty in obtaining suitable public datasets containing sufficient annotated samples for the specific application of glass defect detection. The inherent data imbalance in the industrial context restricted the scope and depth of the training process.

5.3. Future Research Directions

The findings of this project open up several promising avenues for future research and applied development:

- **Real-Time Video Stream Evaluation:** A natural extension would be to test the trained models' performance using a real-time video signal. This would enable the precise evaluation of detection latency and practical precision under continuous operational conditions, moving closer to a live industrial application.
- **Edge Device Deployment Simulation:** It would be highly valuable to deploy the final trained weights onto a resource-constrained edge computing device (e.g., a Raspberry Pi), paired with a camera. This could simulate a real application scenario, allowing for the measurement of the actual processing speed (FPS) achievable on lightweight hardware, thus validating the choice of the Nano variants.
- **User Interface Development:** The creation of a user interface (UI) would enhance the usability of the models. This UI could visualize the results in a user-friendly manner, clearly distinguishing between defect and no defect conditions for production personnel.
- **Industrial Partnership and Validation:** Ideally, these proposed extensions could be executed in collaboration with an industrial partner. Such a partnership would facilitate the evaluation of whether these modern deep learning methods surpass the existing, often complex and expensive, traditional machine learning algorithms currently used for automated quality control in the pharmaceutical sector.

6. Declaration on AI Assistance

The author declares that the empirical research, data analysis, experimental design, and quantitative results presented in this thesis were generated independently and strictly remain the intellectual property of the author.

Artificial intelligence (AI) assistance was utilized solely for the purpose of language refinement, stylistic editing, and optimization of academic reading flow. Specifically, the AI model (Gemini 2.5 Pro) was employed to translate text, correct grammar, and restructure sentences into the required third-person past tense and formal academic style.

No core scientific content, model selection decisions, hyperparameter choices, or interpretation of results were delegated to or generated by the AI model.

7. References

[1] P. Foglia, C. A. Prete, and M. Zanda, “An inspection system for pharmaceutical glass tubes,” *WSEAS TRANSACTIONS on SYSTEMS archive*, vol. 14, pp. 123–136, 2015, Available: https://api.semanticscholar.org/CorpusID:62820567

[2] P. M. Bhatt *et al.*, “Image-based surface defect detection using deep learning: A review,” *Journal of Computing and Information Science in Engineering*, vol. 21, no. 4, p. 040801, Feb. 2021, doi: 10.1115/1.4049535.

[3] N. Buhl, “YOLO object detection explained: Evolution, algorithm, and applications,” *https://encord.com/*. Apr. 2024. Available: https://encord.com/blog/yolo-object-detection-guide/#:~:text=YOLO(YouOnlyLookOnce,identifyobjectsintheimage.

[4] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, “RF-DETR: Neural architecture search for real-time detection transformers.” 2025. Available: https://arxiv.org/abs/2511.09554

[5] Ultralytics, “Object detection datasets overview.” Nov. 2025. Available: https://docs.ultralytics.com/de/datasets/detect/#ultralytics-yolo-format

[6] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, “RF-DETR object detection vs YOLOv12 : A study of transformer-based and CNN-based architectures for single-class and multi-class greenfruit detection in complex orchard environments under label ambiguity.” 2025. Available: https://arxiv.org/abs/2504.13099

[7] capjamesg, “Glass jars defects.” 2024. Available: https://universe.roboflow.com/capjamesg/glass-defect-detection-fvbcu/dataset/3

[8] P. Canoas, “Glass defects detection.” 2025. Available: https://www.kaggle.com/datasets/pedrocanoas/glass-defect?select=dataset

[9] Gemini 2.5 Pro. (n.d.). Google Gemini. Retrieved October 1, 2025, from https://gemini.google.com/app

[10] DeepL Translate: The world’s most accurate translator. (n.d.). https://www.deepl.com/

[11] F. Vollmer, „GitHub Repository: Glass Defect Detection-Evaluating-Object-Models“, GitHub, 1. December 2025. https://github.zhaw.ch/vollmflo/Glass-Defect-Detection-Evaluating-Object-Detection-Models

8. Lists

List of Figures

Figure 1 Comparison Table of different Object Detectors ([4]) 7
Figure 2 Evaluation Metrics ([6]) 8
Figure 3 Raw example images of the dataset A (Ground Truth) 10
Figure 4 Example image of dataset B 12
Figure 5 Example image of dataset B 12
Figure 6 Training Results of Best Weights YOLO 19
Figure 7 Training Results of Best Weights RF-DETR 19
Figure 8 Stratified Validation Comparison 20
Figure 9 Training Results of Best Weights YOLO (Dataset B) 21
Figure 10 Training Results of Best Weights RF-DETR (Dataset B) 22
Figure 11 Validation Comparison 23
Figure 12 Visualization of Stratified Test Results 25
Figure 13 Visualization of Test Results 26
Figure 14 True Positive 28
Figure 15 True Negative 28
Figure 16 False Positive 29
Figure 17 False Negative 29
Figure 18 True Positive 30
Figure 19 False Positive 30

List of Tables

Table 1 Training Duration 18
Table 2 YOLO (Validation) 18
Table 3 RF-DETR (Validation) 19
Table 4 Aggregated Values YOLO 5-Fold 20
Table 5 Aggregated Values RF-DETR 2-Fold 20
Table 6 YOLO (Validation) 21
Table 7 RF-DETR (Validation) 21
Table 8 YOLO (Validation) 22
Table 9 RF-DETR (Validation) 22
Table 10 Test Performance YOLO 5 Fold 24
Table 11 Test Performance RF-DETR 2-Fold 24
Table 12 Testing YOLO (Aggregated) 24
Table 13 Testing RF-DETR (Aggregated) 24

Table 14 Test Performance YOLO 5 Fold 25
Table 15 Test Performance RF-DETR 5-Fold 25
Table 16 Testing YOLO (Aggregated) 26
Table 17 Testing RF-DETR (Aggregated) 26

9. Attachments

9.1. Raw Data

Dataset A

YOLO-Model

Training

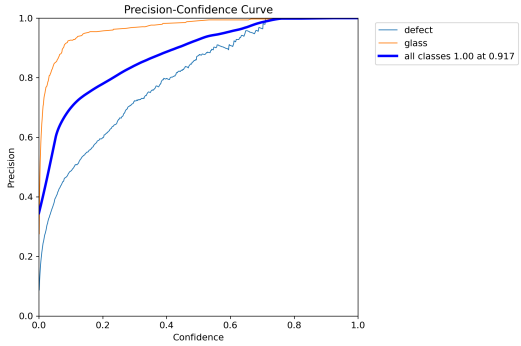


Figure 1: Precision Training

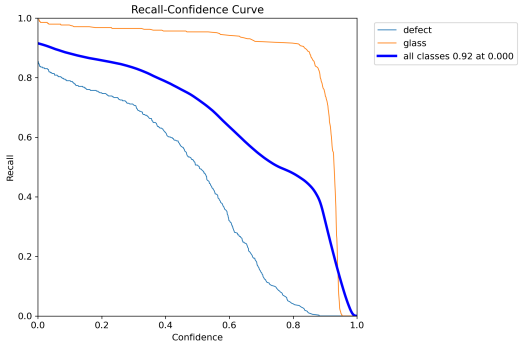


Figure 2: Recall Training

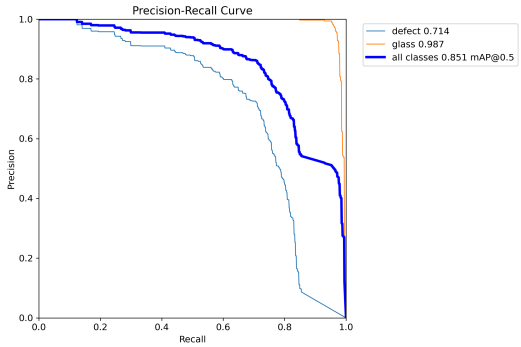


Figure 3: Precision/Recall Training

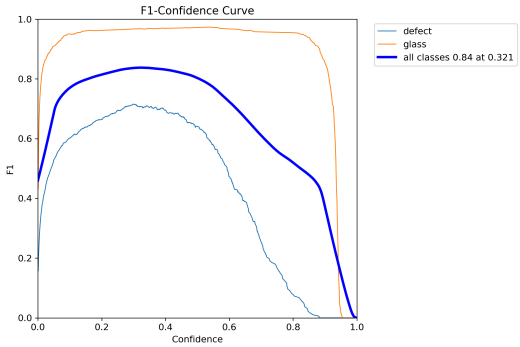


Figure 4: F1-Score Training

Testing

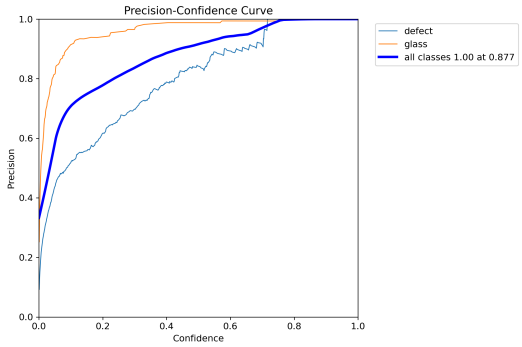


Figure 5: Precision Test

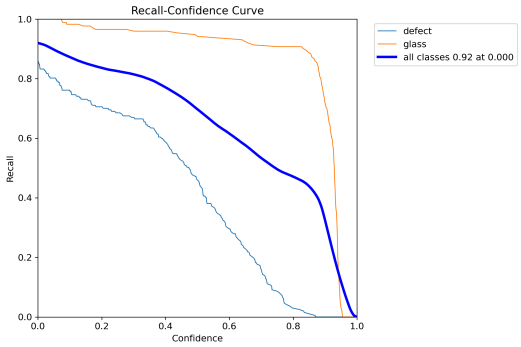


Figure 6: Recall Test

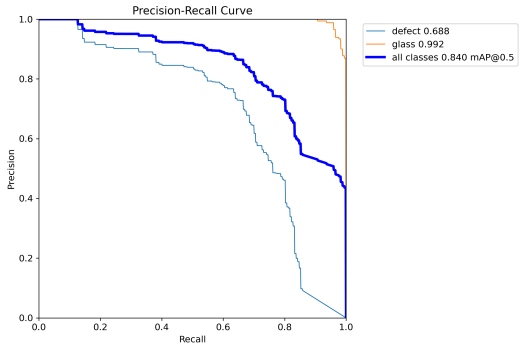


Figure 7: Precision/Recall Test

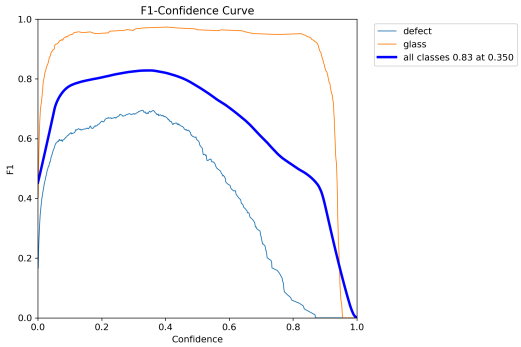


Figure 8: F1-Score Test

RF-DETR

Training

See Model Results of Training Figure 7

Testing

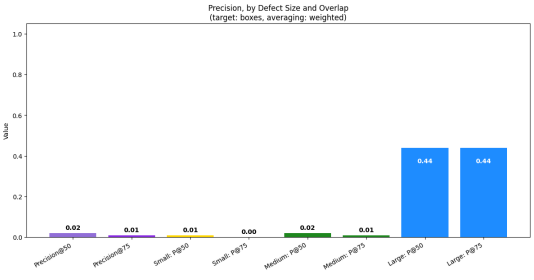


Figure 9: Precision Testing

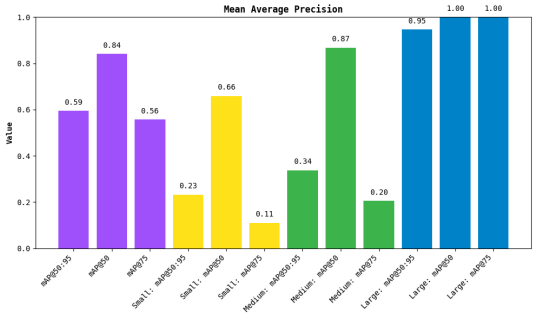


Figure 10: mAP Testing

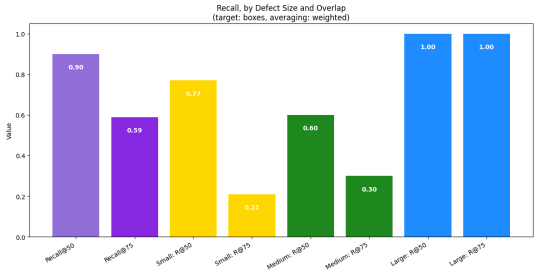


Figure 11: Recall Testing

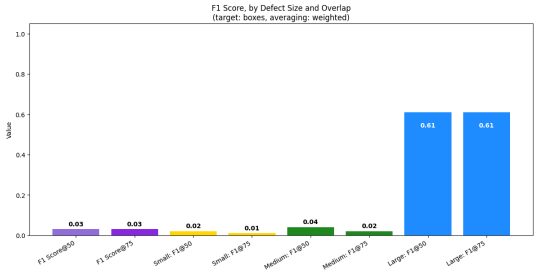


Figure 12: F1-Score Testing

Dataset B

YOLO-Model

Training

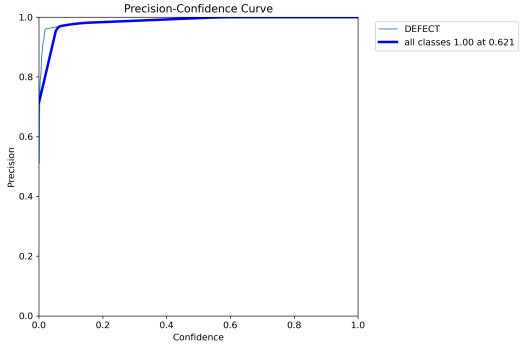


Figure 13: Precision Training

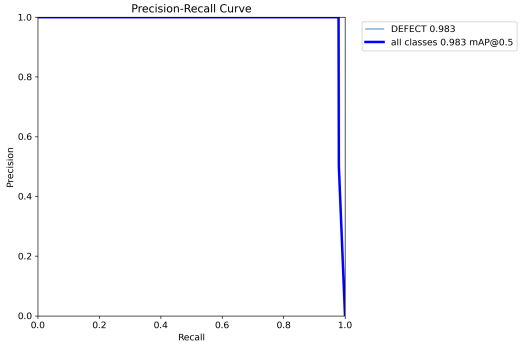


Figure 14: Precision/Recall Training

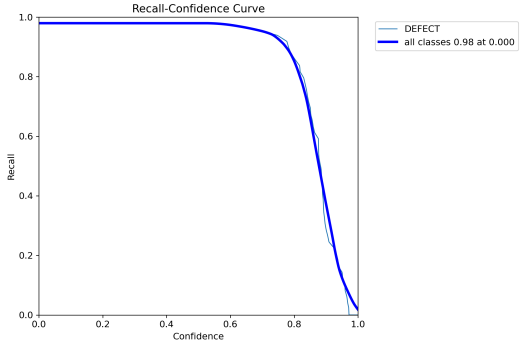


Figure 15: Recall Training

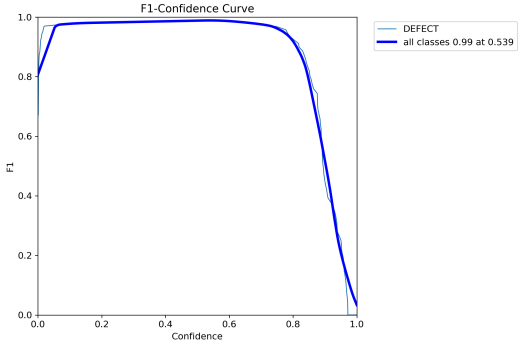


Figure 16: F1-Score Training

Testing

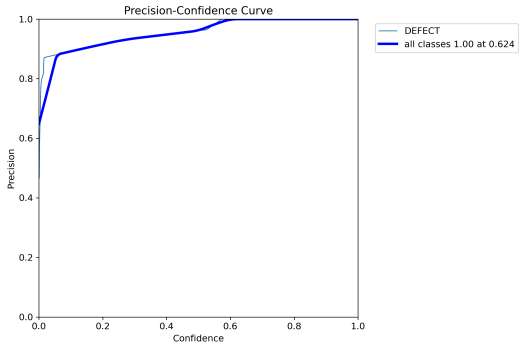


Figure 17: Precision Testing

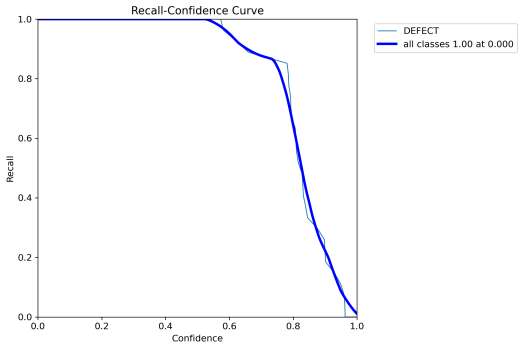


Figure 18: Recall Testing

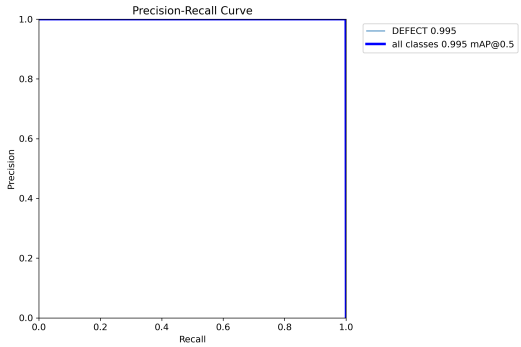


Figure 19: Precision/Recall Testing

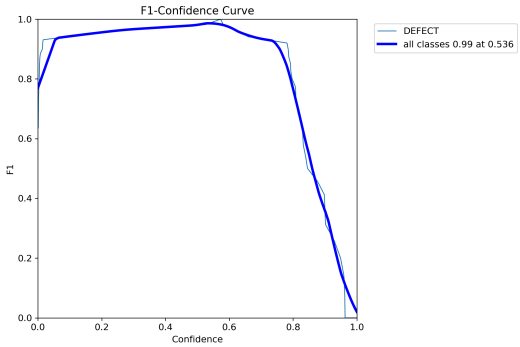


Figure 20: F1-Score Testing

RF-DETR

Training

See Model Results of Training Figure 10

Testing

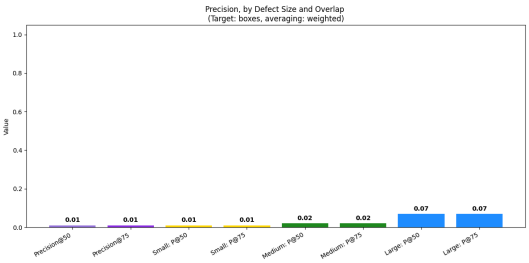


Figure 21: Precision Testing

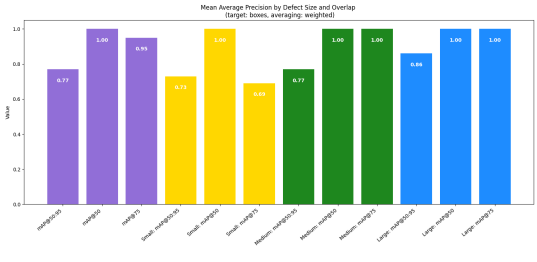


Figure 22: mAP Testing

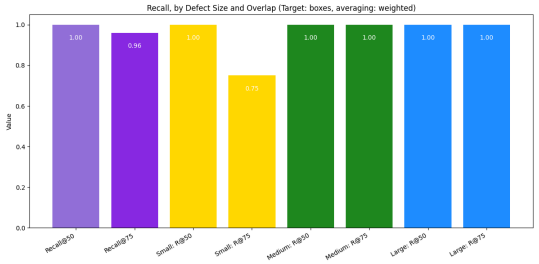


Figure 23: Recall Testing

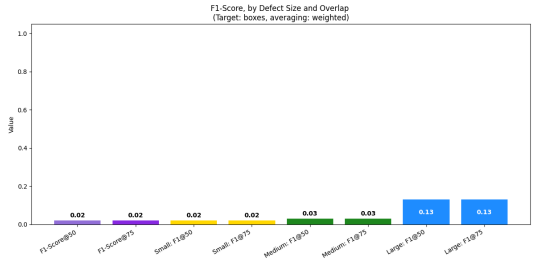


Figure 24: F1-Score Testing

Sample Images

Dataset A

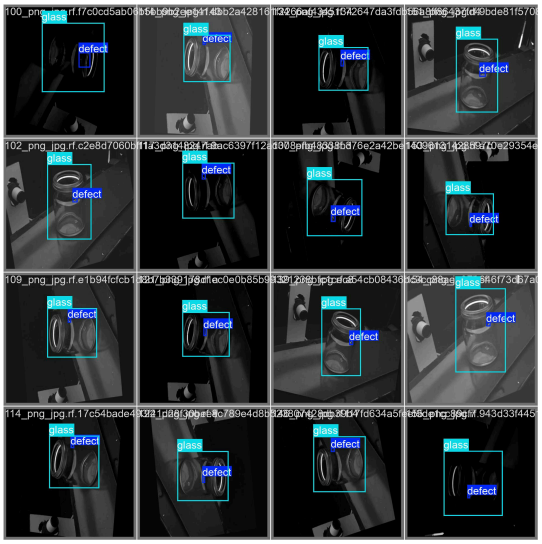


Figure 25: Ground Truth

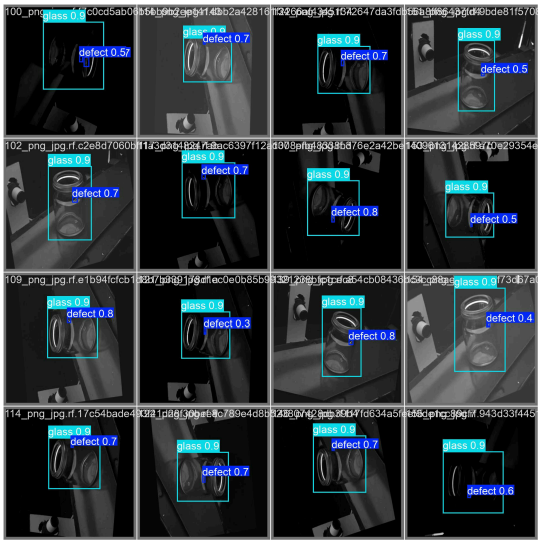


Figure 26: Predictions

Dataset B

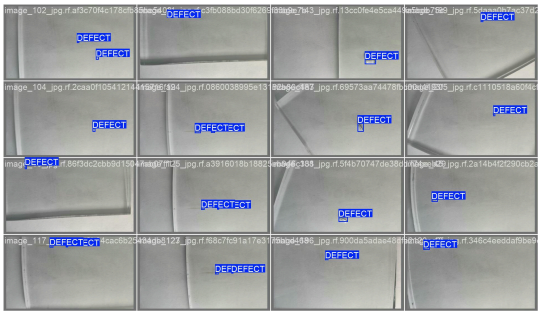


Figure 27: Ground Truth

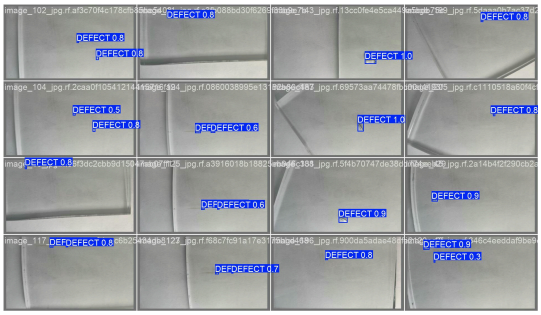


Figure 28: Predictions